

# Beyond Data: Solving Differential Equations with Physics-Informed Neural Networks

Jenson P Wightman  
Natural Sciences Tripos, Part II Physics  
University of Cambridge

May 4, 2026

## Abstract

Solving ordinary and partial differential equations is the cornerstone of computational physics, traditionally relying on numerical solvers such as finite difference methods. In an era of unprecedented computational power, machine learning offers a new paradigm: Physics-Informed Neural Networks (PINNs), which encode governing equations directly into a neural network's training. This project evaluates PINNs against traditional numerical methods, finding them orders of magnitude slower for standard forward problems but offering genuine advantages for inverse problems with unknown parameters or incomplete data. PINN structure and activation function choice prove to be critical; a viscosity-dependent crossover is observed in which smooth solutions favour tanh while near-shock solutions favour sinusoidal activations, demonstrating that optimal network design is solution-dependent. Failure modes, including trivial collapse and spectral bias, are explored throughout.

# Contents

|          |                                                                                 |           |
|----------|---------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                             | <b>4</b>  |
| <b>2</b> | <b>Theoretical Background</b>                                                   | <b>4</b>  |
| 2.1      | Neural Network Structure and Training . . . . .                                 | 4         |
| 2.2      | Loss in Physics-Informed Neural Networks . . . . .                              | 5         |
| 2.3      | Optimisation . . . . .                                                          | 5         |
| 2.3.1    | Adam Optimiser . . . . .                                                        | 6         |
| 2.3.2    | L-BFGS Optimiser . . . . .                                                      | 6         |
| 2.4      | Activation Functions in Neural Networks . . . . .                               | 7         |
| 2.5      | Solution Quality Metrics . . . . .                                              | 8         |
| <b>3</b> | <b>Results &amp; Discussion</b>                                                 | <b>9</b>  |
| 3.1      | Initial Tests . . . . .                                                         | 9         |
| 3.1.1    | Exponential Decay ODE . . . . .                                                 | 9         |
| 3.1.2    | Damped Harmonic Oscillator . . . . .                                            | 10        |
| 3.1.3    | Effect of Collocation Density . . . . .                                         | 11        |
| 3.1.4    | Spectral Bias in PINNs . . . . .                                                | 11        |
| 3.2      | Comparison with Numerical Methods . . . . .                                     | 13        |
| 3.2.1    | Solving 1D Diffusion . . . . .                                                  | 13        |
| 3.2.2    | The Inverse Problem . . . . .                                                   | 15        |
| 3.2.3    | 1D Burgers' equation . . . . .                                                  | 16        |
| 3.2.4    | Curriculum Learning . . . . .                                                   | 18        |
| <b>4</b> | <b>Conclusion</b>                                                               | <b>19</b> |
| <b>A</b> | <b>1D Diffusion timing methodology</b>                                          | <b>20</b> |
| <b>B</b> | <b>Derivation of the Newton update[2]</b>                                       | <b>21</b> |
| <b>C</b> | <b>1D Burger's Equation Viscosity sweep, solution and error grid</b>            | <b>22</b> |
| <b>D</b> | <b>1D Burger's Equation curriculum viscosity sweep, solution and error grid</b> | <b>23</b> |

# Preliminaries

## Notation and Terminology

This report uses  $u(\underline{x}, t)$  to represent the exact solution of a PDE in a domain. PINN-calculated solutions are represented by  $\hat{u}(\underline{x}, t; \theta)$ .

For the avoidance of confusion,  $\hat{L}[f(\underline{x}, t)]$  is occasionally used to represent a differential operator acting on a function  $f(\underline{x}, t)$ .

L2RE or ‘Error’ typically refers to relative  $L^2$  error as defined in Section 2.5.

Where measurements of the compute time for an experiment are taken, wall-clock time is the metric used as given by `time.perf_counter()`. I occasionally refer to this colloquially as ‘runtime’ but wall-clock time is implied.

## Acknowledgements

I would like to thank both Christopher M. Bishop [1] and Ian Goodfellow et al. [2] for their books *Pattern Recognition and Machine Learning* and *Deep Learning* respectively, for providing a great deal of my understanding and theoretical background for this project. I also thank Sangheon Lee[3] for his website and tutorial on PINNs for aiding my initial understanding of the PyTorch implementation

## Hardware

All neural network training and numerical solvers were run on a Lenovo ThinkPad X1 Carbon Gen8 specifications:

- Intel Core i7-10610U CPU @ 1.80 GHz (2.30 GHz)
- 16 GB RAM

GPU acceleration was not used for any neural network training in the project. Thus, any reported training times are likely longer than they would be in more common applications. GPU acceleration would likely disproportionately benefit PINN training relative to numerical solvers, so the slowdown factors presented in this report should be interpreted as upper bounds rather than absolute method differences.

# 1 Introduction

Numerical methods underpin the entirety of modern computational physics. Techniques have been refined for decades, with well-understood mathematical theory, convergence rates and failure modes. Such has been the success of computational numerical methods, there has been little motivation to consider alternatives.

Recent advances in deep learning have introduced a new approach. Physics-Informed Neural Networks [4] encode governing equations directly into a neural network’s training: rather than discretising the spatial-temporal domain into a grid, the network learns a continuous representation of the solution by minimising the PDE residual at scattered collocation points. This continuous parametrisation, combined with the network’s many degrees of freedom and natural integration of partial observational data, positions PINNs as a potentially powerful tool for problems that traditional solvers are less adept at.

This project provides a systematic evaluation of PINNs across a hierarchy of problems: from analytical ordinary differential equations through linear partial differential equations to the non-linear viscous Burgers’ equation. PINN solutions are compared against both analytical solutions (where suitable) and traditional numerical methods. Three strands of investigation run throughout: the basic capabilities and computational scaling of PINNs, the suitability of different network architectures and activation functions, and the regimes in which numerical methods or PINNs are advantageous.

This report aims to establish where PINNs sit as a computational tool and for what purposes they are most useful. Generally, we see PINNs are dramatically slower than traditional methods for standard forward problems, but offer advantages for inverse problems and data assimilation.

## 2 Theoretical Background

### 2.1 Neural Network Structure and Training

As the name suggests, PINNs are a subset of neural networks. In physical problems, we want to find the relationship between a set of coordinates (e.g.  $x, t$ ) and the output  $u(x, t)$ ; for this reason, the most readily used form of neural network in a Physics-Informed Neural Network is a so-called feed-forward Multi-Layer Perceptron (MLP). ‘Feed-forward’ in a neural network is the equivalent of our intuition for mapping coordinates to outputs — no internal feedback is needed. Such neural networks are commonly described as a universal function approximator.

The architecture consists of an input layer, several ‘hidden’ layers and an output layer. The association between the input to a layer and its output are described by a set of parameters (the weights and biases, denoted together as  $\theta$ ) and a non-linear activation function. Symbolically:

$$\mathbf{z}^{(l)} = \sigma(\mathbf{W}^{(l)} \cdot \mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}) \quad (1)$$

where  $\mathbf{z}^{(l)}$  is the layer output,  $\mathbf{W}^{(l)}$  is the weight matrix for layer  $l$ ,  $\mathbf{b}^{(l)}$  is the bias vector for layer  $l$ , and  $\sigma(\cdot)$  is the non-linear activation function [1].  $\sigma(\cdot)$  must be non-linear to create the ‘expressibility’ of a neural network; a network with linear activation would collapse to a single linear transformation. The output of such a network is thus  $\hat{u}(x, t; \theta)$ , and it follows that by tuning the parameters ( $\theta$ ) of the network, one could converge on a neural network

that produces the desired solutions to a given problem; but how do we refine the parameters of this neural network?

## 2.2 Loss in Physics-Informed Neural Networks

Intuitively, to improve the predictive performance of a neural network it is pertinent to minimise the difference between PINN output and known values or constraints; usually the training data.

This is where the ‘Physics-Informed’ arises in ‘Physics-Informed Neural Networks’. Unlike purely data-driven training (which isn’t always possible), one can implement physical laws into the network’s training. With this in mind, we define three components of the loss function for the standard PINN formulation:

$$\mathcal{L}(\theta) = \lambda_{\text{IC}} \mathcal{L}_{\text{IC}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}} + \lambda_{\text{physics}} \mathcal{L}_{\text{physics}} \quad (2)$$

Weightings ( $\lambda_i$ ) of the loss components in Eq. (2) are employed to ensure that one loss component does not dominate over another since each component ( $\mathcal{L}_i$ ) can have different magnitudes.

The initial/boundary condition losses (IC/BC terms) are given by the mean square errors of PINN output and the respective known values, i.e.

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_I} \sum_{i=1}^{N_I} (\hat{u}(x_i, 0; \theta) - u(x_i, 0))^2 \quad (3)$$

where  $N_I$  is the number of sample points along  $t = 0$ , and likewise for the BC losses.

Similarly, the ‘physics loss’ can be defined as the mean square residual of the PDE, which is made particularly simple by writing the PDE in the form

$$\hat{L}[u(x, t)] = 0 \quad (4)$$

The mean square residual can then be calculated across  $N_c$  collocation points in the domain:

$$\mathcal{L}_{\text{physics}} = \frac{1}{N_c} \sum_{i=1}^{N_c} \left( \hat{L}[\hat{u}(x_i, t; \theta)] \right)^2 \quad (5)$$

Traditional neural network training also invokes a data loss term:

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} (\hat{u}(x_i, t_i; \theta) - u_{\text{data}, i})^2 \quad (6)$$

where the PINN solution  $\hat{u}(x_i, t_i; \theta)$  is evaluated at the coordinates of the given training data. A data loss is not necessary in this project, except in the case of inverse problems.

## 2.3 Optimisation

Now with a relevant metric ( $\mathcal{L}(\theta)$ ) to optimise the PINN with respect to, we need a mechanism for this optimisation.

Let us first consider a simple approach to optimisation: gradient descent. The loss  $\mathcal{L}$  is a number that depends on every weight and bias in the network. For simplicity we will refer to the weights and biases as the ‘parameters’ ( $\theta$ ). PyTorch [5] is able to calculate the gradient  $\frac{\partial \mathcal{L}}{\partial \theta}$  using `torch.autograd`, allowing us to implement a simple update rule:

$$\theta' = \theta - \eta \cdot \frac{\partial \mathcal{L}}{\partial \theta} \quad (7)$$

Thus a positive gradient (increasing weight  $\implies$  increased loss) will decrease the weight and vice versa. Performing this for all parameters and repeating for many cycles (or epochs) should converge on a sufficiently optimised set of weights and biases. There are, however, more efficient and effective optimisation mechanisms that are implemented in this report. The two optimisers used throughout are the Adam optimiser and the L-BFGS optimiser.

### 2.3.1 Adam Optimiser

The Adam optimiser (Adaptive Moment Estimation) [6] aims to solve two main issues with standard gradient descent: all weights have the same learning rate, and the update has no memory of previous steps. Intuitively, if a parameter has been updated largely in the same direction for several steps previous, a single noisy gradient should not deter from this. To implement this, Kingma and Ba define:

$$m^{(t+1)} = \beta_1 \cdot m^{(t)} + (1 - \beta_1) \cdot \nabla_{\theta} \mathcal{L}(\theta_t) \quad (8)$$

The hyperparameter  $\beta_1$  determines the previous gradient content of current momentum,  $m^{(t)}$ . Adam also keeps a ‘running average’ of the squared gradients for each weight,  $v^{(t)}$ :

$$v^{(t+1)} = \beta_2 \cdot v^{(t)} + (1 - \beta_2) \cdot (\nabla_{\theta} \mathcal{L}(\theta_t))^2 \quad (9)$$

where the gradient squared is to be taken element-wise. For completeness, it should also be noted that the  $m$  and  $v$  values are bias corrected for their small initial values using

$$\hat{m} = \frac{m}{1 - \beta_1^t} \quad \hat{v} = \frac{v}{1 - \beta_2^t} \quad (10)$$

where  $t$  is the step number. Now similarly to eq.7, one updates the weights using:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \cdot \frac{\hat{m}^{(t)}}{\sqrt{\hat{v}^{(t)}}} \quad (11)$$

### 2.3.2 L-BFGS Optimiser

The Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS)[7] algorithm is a quasi-Newton method. Ideally, one could compute the exact Hessian ( $\mathbf{H}$ ) in the parameter space and use the update rule[2](see Appendix B for a short proof):

$$\theta^{(t+1)} = \theta^{(t)} - \mathbf{H}^{-1} \cdot \nabla_{\theta} \mathcal{L} \quad (12)$$

thus taking a single step in a quadratic (i.e. near the minima) loss surface towards the minima. This has a clear issue that the Hessian  $\left( H_{ij} = \frac{\partial^2 \mathcal{L}}{\partial \theta_i \partial \theta_j} \right)$  has  $N^2$  entries for a neural

network of  $N$  parameters. Computing the Hessian and its inverse is prohibitively expensive. Instead of computing the Hessian and its inverse, the BFGS algorithm approximates the inverse Hessian using the difference of the parameters ( $s_i$ ) and the difference of the loss gradient ( $y_i$ ). This has removed the inversion calculation, but is still maintaining an  $N \times N$  matrix.

L-BFGS stores only the last  $m$  pairs of  $(s_i, y_i)$  and reconstructs the action of the inverse Hessian from these pairs, never forming the full matrix (hence 'Limited-memory'). The L-BFGS algorithm has complexity  $\mathcal{O}(md)$  (for small  $m$ ) [8] instead of  $\mathcal{O}(d^2)$ . A more comprehensive treatment of L-BFGS theory is beyond the scope of this report; see ref.[7] for details.

Standard practice in PINN training is to implement both Adam and L-BFGS training epochs for optimal convergence. Adam optimisation is used during initial epochs where its adaptive learning rates excel at taking steps toward the minima in a noisy loss surface. L-BFGS optimisation is used in later epochs for refinement of the neural network, where the loss surface is approximately quadratic and quasi-Newton methods work well. [9].

## 2.4 Activation Functions in Neural Networks

As briefly discussed earlier, activation functions in a neural network are non-linear functions applied to the output of a layer before input to the next layer. Non-linear activations prevent the neural network from collapsing into a single linear transformation, but also have greater implications for the applicability of neural networks to certain problems. It is known that to solve a  $k$ -th order PDE, the  $k$ -th derivative of the activation function should be bijective [10] (that is, essentially, one-to-one). For periodic activations, this condition is satisfied locally within the range of pre-activations.

In this project, the PINN class in models.py supports several activation functions: ReLU, tanh, swish (or SiLU in PyTorch) and sin. All of these are provided as built-in nn.Module classes in PyTorch apart from sin activation, so we define a custom Sin class :

```
class Sin(nn.Module):
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.sin(x)
```

We also implement a specific architecture type that utilises sin activation, so-called Sinusoidal Representation Networks (SIRENS), following the original paper by Sitzmann et al (2020) [11] and their GitHub associated with the paper [12]. The SIREN implementation is found in src/models.py and included below for completeness:

```
class SIRENLayer(nn.Module):

    def __init__(self, in_features: int, out_features: int,
                 omega_0: float = 30.0, is_first: bool = False):
        super().__init__()
        self.omega_0 = omega_0
        self.linear = nn.Linear(in_features, out_features)
        self._init_weights(in_features, is_first)

    def _init_weights(self, in_features: int, is_first: bool):

        with torch.no_grad():
            if is_first:
```

```

        bound = 1.0 / in_features
    else:
        bound = math.sqrt(6.0 / in_features) / self.omega_0
    self.linear.weight.uniform_(-bound, bound)
    nn.init.zeros_(self.linear.bias)

def forward(self, x: torch.Tensor) -> torch.Tensor:

    return torch.sin(self.omega_0 * self.linear(x))

```

Where  $\omega_0$  scales the pre-activations to span multiple periods of the sine function, and weights are initialised according to bounds defined separately for the first and hidden layers. This maintains activation stability, instead of weights exploding or vanishing. SIRENs are designed to better represent high-frequency content, potentially offering clear advantages for PINN applications, where they may provide a richer representation of challenging boundaries or sharp gradients, which are prevalent in physical situations.

## 2.5 Solution Quality Metrics

While the loss is an excellent metric to optimise the neural network against, it can be a poor measure of solution quality.

Indeed, there is usually correlation between low loss and a ‘good’ solution; however this need not always be the case. For instance, in the case of trivial collapse (observed later) the PINN can produce a small loss but a clearly wrong solution. Loss is also not directly comparable across experiments due to different weightings or problem parameters. For these reasons, we implement a selection of situation-specific solution quality metrics.

In the case of experiments with exact solutions, we report relative  $L^2$  error (Eq. 13) against the analytical solution, computed on a fixed evaluation grid of  $N$  points:

$$\text{Relative } L^2 \text{ Error} = \frac{\sqrt{\sum_{i=1}^N (u(x_i, t_i) - \hat{u}(x_i, t_i; \theta))^2}}{\sqrt{\sum_{i=1}^N (u(x_i, t_i))^2}} \quad (13)$$

Measuring the error in this way has the benefit of being dimensionless and scale-invariant. It should be noted that, where reasonable, the number of evaluation points is kept the same for comparison, but in some cases, such as high-frequency solutions or differing dimensionality, a larger  $N$  is required. Regardless, variation in the error due to  $N$  is negligible for large  $N$ .

For experiments with non-analytical solutions that can be solved to high precision via numerical methods, we also implement relative  $L^2$  error with the approximation that  $u_{\text{numerical}}(x_i, t_i) = u(x_i, t_i)$ .

For more complex cases, physical conservation laws can be implemented as a check of solution quality, such as conservation of energy, probability, etc.



### 3 Results & Discussion

#### 3.1 Initial Tests

As a first validation of a PINN in this project, we apply a PINN to simple analytical ODEs to explore the basic properties and computational complexity scalings of the application of physics-informed neural networks, as well as demonstrating certain capabilities and difficulties associated with PINNs, before expanding on such results in later sections.

##### 3.1.1 Exponential Decay ODE

We first solve the simple ODE for exponential decay:

$$\frac{du}{dt} = -\lambda u, \quad u(0) = u_0 \quad (14)$$

with  $\lambda = 1.0$  and  $u_0 = 1.0$  over  $t \in [0, 5]$ . The analytical solution to eq. (14) is  $u(t) = u_0 e^{-t}$ .

A baseline  $2 \times 32$  [layers  $\times$  neurons] tanh network was trained with 200 collocation points over 5500 epochs (5000 Adam + 500 L-BFGS epochs) and plotted against the analytical solution. The training loss history is also plotted; see Figure 1.

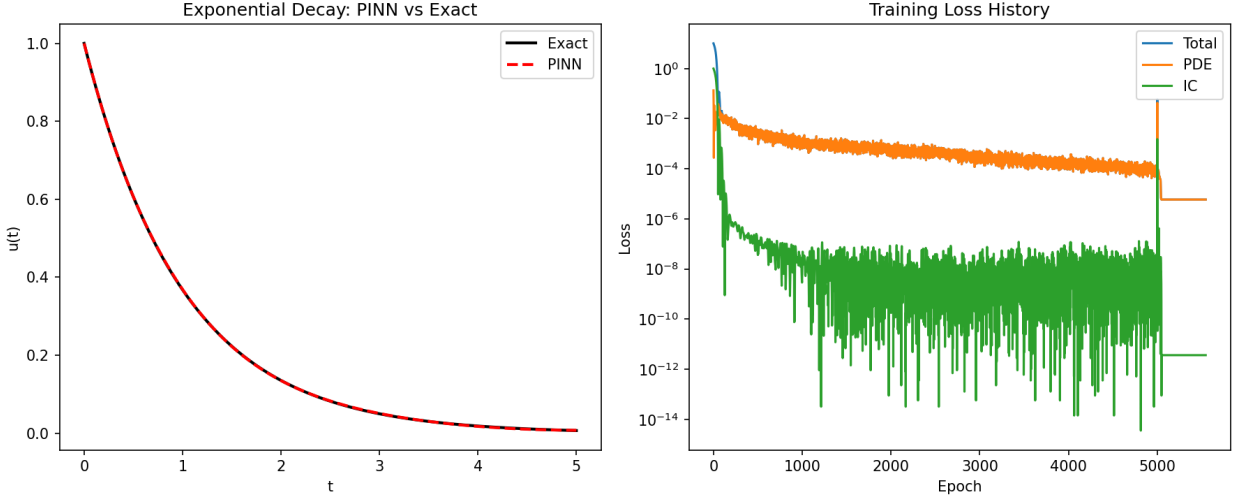


Figure 1: (a) Exact and PINN-predicted exponential decay solution. (b) Training loss history for the baseline training.

Clearly the PINN performs excellently for a simple ODE, as expected. We also see that the loss converges rapidly and, importantly, the L-BFGS epochs perform well at optimising the neural network further, signified by the dip and plateau in error at the 5000 epoch mark.

Now, by initiating the PINN with three distinct architectures:  $[1 \times 16]$ ,  $[2 \times 32]$ ,  $[4 \times 64]$  we demonstrate the comprehensive ability of such PINNs with respect to eq.14 and the scaling of their compute time. The relative  $L^2$  error and training time required for each architecture are plotted. Unsurprisingly, for such a simple ODE, one sees that the error does not change much with more complex architectures, suggesting that even the  $[1 \times 16]$  is readily able to solve the exponential decay equation. However, the data gathered does show that there is a growing computational cost with architecture size (see Figure 2).

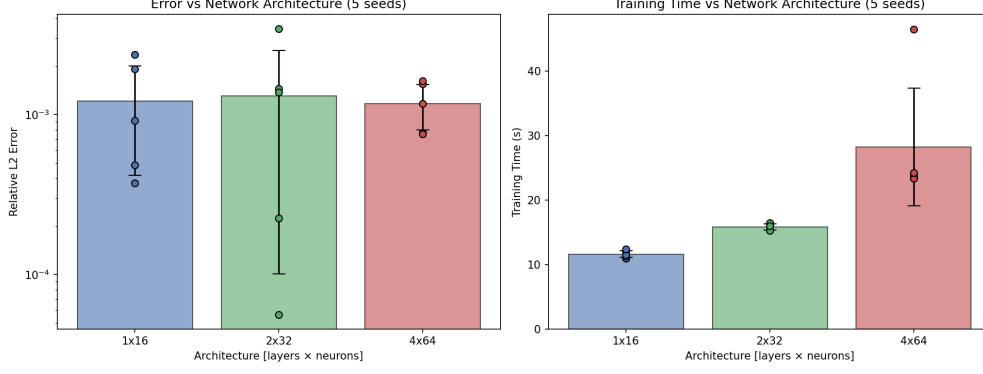


Figure 2: (a) Relative  $L^2$  error for PINN solutions to the exponential decay ODE for three different network complexities. (b) Total training time of the PINN on the simple ODE for the three architectures.

### 3.1.2 Damped Harmonic Oscillator

The same investigations of PINN solutions to the exponential decay equation were performed, using the same training variables and three architectures, on a damped harmonic oscillator:

$$\frac{d^2x}{dt^2} + 2\gamma \frac{dx}{dt} + \omega_0^2 x = 0 \quad (15)$$

with an analytical solution

$$x(t) = e^{-\gamma t} \left[ x_0 \cos(\omega_d t) + \frac{v_0 + \gamma x_0}{\omega_d} \sin(\omega_d t) \right], \quad \omega_d \equiv \sqrt{\omega_0^2 - \gamma^2} \quad (16)$$

For the simplest ODE, seen in Section 3.1.1, the solution is easily comprehended by a basic PINN. A slightly more complex ODE is required to produce the expected decrease in error with network complexity. For a damped harmonic oscillator, this is indeed what we see (see Figure 3). Error decreases from approximately  $2 \times 10^{-2}$  for  $[1 \times 16]$  to  $4 \times 10^{-3}$  for the  $[4 \times 64]$  architecture. Training times also increased significantly, indicative of the necessarily more complex nature of the neural network.

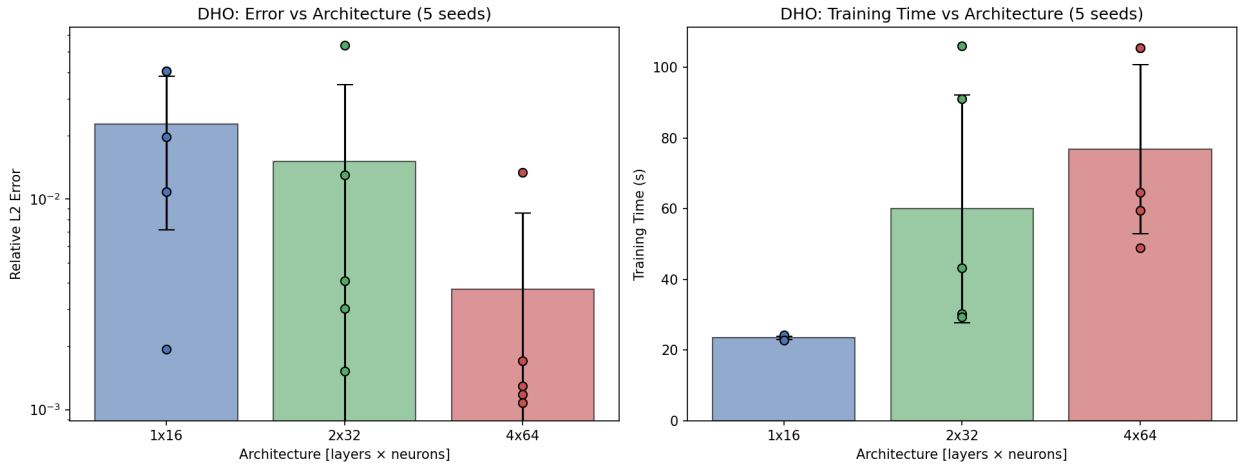


Figure 3: (a) Relative  $L^2$  error for PINN solutions to the damped harmonic oscillator ODE for three different network complexities. (b) Total training time of the PINN on the DHO ODE for the three architectures.

As seen in Figures 2 and 3, evaluating the ODEs across five random weight initialisations produces high variance in both relative  $L^2$  error and training time. This can be attributed to the random initialisation of the neural network’s weights, and so a ‘lucky’ seed converges rapidly to the solution, while an ‘unlucky’ seed is trapped in local minima of the solution space, skewing the standard deviation.

### 3.1.3 Effect of Collocation Density

To further quantify the parameters affecting PINN error, we examine L2RE across varying collocation point density ( $N \in \{10, 50, 200, 1000\}$ ) for both eq. 14 and eq. 15.

For both the DHO equation and exponential decay, it is clear that a larger  $N$  significantly reduces the PINN solution error, until a point. For a sufficiently large  $N$ , the PINN effectively has all the information it ‘needs’, and so more collocation points do not significantly improve error. It is also clear that more complex solutions require a greater collocation density. Notice that relative  $L^2$  error for the DHO equation only matches the error for the exponential decay solution at  $N = 1000$ . With this in mind, the remainder of this report uses  $N = 1000$  for ODEs and  $N = 2000$  for PDEs to accommodate the 2D input space.

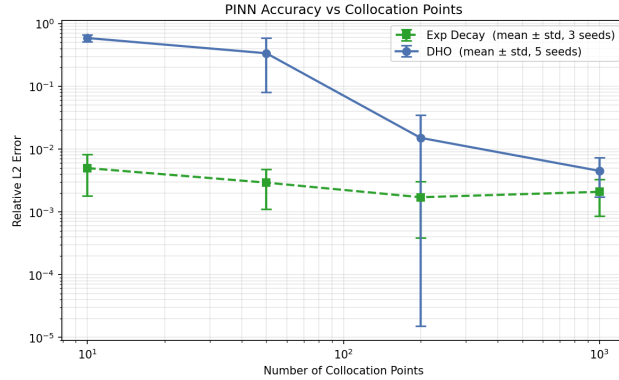


Figure 4: L2RE error against number of collocation points for two PINN (tanh activated) trained on the DHO and exponential decay equation across 5000 Adam + 500 L-BFGS epochs.

### 3.1.4 Spectral Bias in PINNs

PINNs are known to ‘struggle’ with certain problems more than others. A notorious example of such is so-called ‘spectral bias’ [13]. Spectral bias is the tendency of neural networks to learn lower-frequency components faster than higher-frequency modes. In the context of PINNs, this often manifests as the biased learning of certain structures in the solution. For instance, a PINN may more readily learn smooth global structures without capturing more distinct phenomena such as boundary layers or other sharp gradients.

As a brief demonstration, let us consider a ‘high-frequency’ SHO for which the DHO class with  $\gamma = 0$  is re-used. We first plot the PINN-generated solution for the SHO for three different activation functions: sine, tanh, and ReLU.

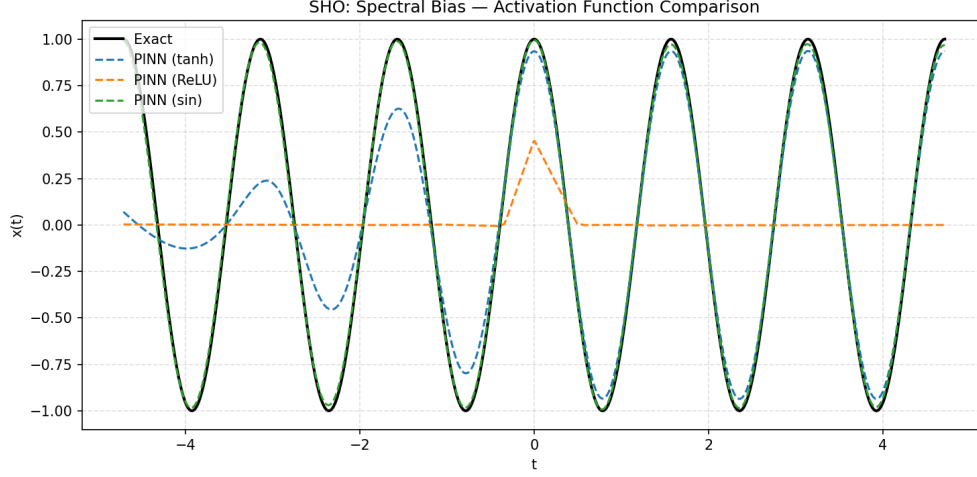


Figure 5: PINN-generated solutions to the SHO ODE for sine, tanh, and ReLU activations, each trained across 10,000 Adam epochs and 1,000 L-BFGS epochs.

As expected, ReLU activation fails to learn any meaningful solution to the ODE because its second derivative is zero almost everywhere, providing no useful gradient for second-order PDE residuals. The tanh activation that we have utilised so far handles this relatively high-frequency solution well, but does worsen for  $t < 0$ , potentially due to random weight initialisation presenting a biased direction for learning. The sine activation performs almost flawlessly here; I do note that it is in a sense, ‘cheating’ since the solution is itself a sine and thus the network effectively only needs to optimise the amplitude and phase of the sine to match the solution. Nevertheless, it is qualitatively clear that certain activation functions improve the spectral bias of PINNs.

For a more quantitative analysis of spectral bias, a PINN (sine activated) is trained on eq.15 for a range of frequencies (5-50 Hz) and L2RE measured at each point, see Figure 6.

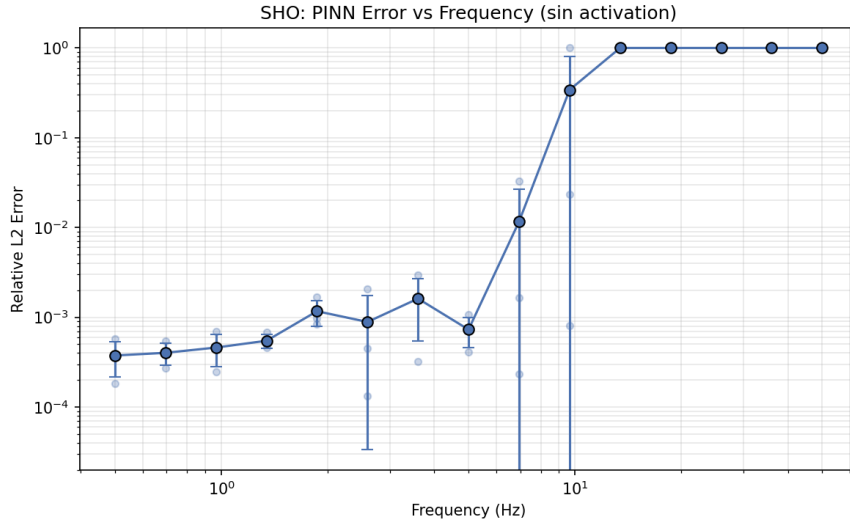


Figure 6: Frequency sweep  $\nu \in [0.5, 50]$  Hz for a  $[3 \times 64]$  network with sine activation, trained across 10,000 epochs with 1,000 L-BFGS epochs. The PINN is seeded three times at each frequency

Relative  $L^2$  error is significantly reduced for low frequencies and approaches unity for high

frequencies. The likely reason for this observed saturation at unity is 'trivial collapse'. At the highest frequencies, the PINN more readily solves the trivial case of eq.15,  $x(t) = 0$ . The loss of the trivial solution rapidly approaches 0, but its L2RE tends to 1, as is clear in eq.13.

We have now seen the basic properties and scaling of PINNs in terms of their architecture, as well as the presence of spectral bias as an inherent issue in PINNs and how different activation functions affect their ability to comprehend high-frequency or sharp components of solutions.

## 3.2 Comparison with Numerical Methods

PINNs inherently have a significant degree of complexity associated with them. Creating the neural networks themselves involves a reasonable degree of complexity, though the PyTorch library [5] has already done much of the work for us, but, more crucially, the training of neural networks is a computationally expensive task requiring both significant resources and time. Many tasks do not require the complexity that neural networks possess and are solved more efficiently by existing targeted solutions e.g numerical methods. However, PINNs lend themselves well to inference from data, aided by their many degrees of freedom and physics-informed losses.

### 3.2.1 Solving 1D Diffusion

We now compare PINN solutions with a numerical solution for the 1D diffusion equation:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2} \quad (17)$$

with initial condition:

$$u(x, 0) = \sin\left(\frac{n\pi x}{L}\right) \quad (18)$$

For all cases used in this section, the  $n = 1$  mode is used. With Eq. (18), the 1D diffusion equation has an exact solution:

$$u_{\text{exact}}(x, t) = \sin(kx) e^{-\alpha k^2 t}, \quad k = \frac{n\pi}{L} \quad (19)$$

Numerical methods for solving Eq. (17) are well known and we implement the Forward Time Central Space (FTCS) method here [14]. The update rule for FTCS is:

$$u_i^{n+1} = u_i^n + r (u_{i+1}^n - 2u_i^n + u_{i-1}^n), \quad r = \frac{\alpha \Delta t}{\Delta x^2} \quad (20)$$

Training a PINN on Eq. (17) and solving it using FTCS produced the solutions shown in Figure 7.

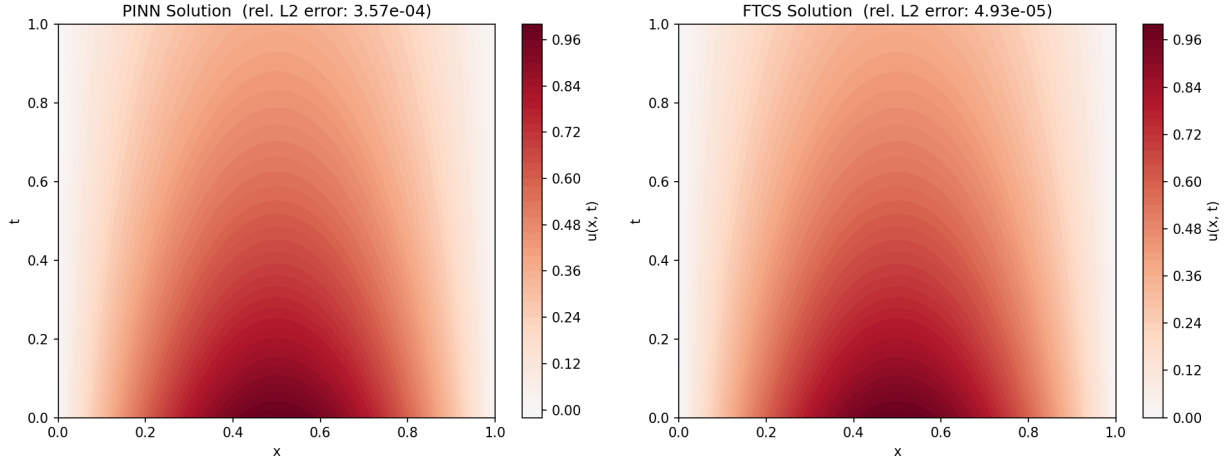


Figure 7: Solutions to the 1D diffusion equation computed using a PINN  $[3 \times 64]$  and the FTCS method with a diffusivity  $\alpha = 0.1$  and homogeneous Dirichlet boundary conditions. The PINN was trained over 10,000 Adam + 1,000 L-BFGS epochs.

For this use case, both perform well, with FTCS producing an order of magnitude lower error, but how do their compute times compare? To test this, the FTCS solver was run again and the PINN retrained.

An initial comparison (see Appendix A) using Adam-only training with a stopping criterion at the FTCS error level proved unsuitable: the Adam optimiser exhibits large error fluctuations near convergence, producing unreliable stopping behaviour, and Adam alone struggles to reach FTCS-level accuracy within a reasonable compute budget. We therefore adopted the following two-stage timing protocol:

- Adam epochs until the relative  $L^2$  error falls below  $90\times$  the FTCS error (max 30,000 epochs; evaluation interval 200 epochs).
- L-BFGS refinement until error equals  $10\times$  the FTCS error (max 5,000 epochs; evaluation interval 50 epochs).

The method is clear in Figure 8, where relative  $L^2$  error is plotted against wall-clock time. The fluctuations associated with Adam optimisation are evident in the first stage, as is the greater compute cost associated with L-BFGS, demonstrated by a shallower gradient in this space.

Table 1: 1D Diffusion: FTCS vs PINN Timing Comparison ( $\alpha = 0.1$ , mode  $n = 1$ )

| Method | Architecture                  | Rel. L2 Error | Time (s) |
|--------|-------------------------------|---------------|----------|
| FTCS   | $\Delta x = 0.01$ , $r = 0.4$ | 4.92e-05      | 0.0161   |
| PINN   | $3 \times 64$ tanh            | 4.79e-04      | 205.2214 |

*Note: PINN training utilized 4,000 Adam + 750 L-BFGS steps (time excludes evaluation overhead). PINN is  $12,782\times$  slower than FTCS at comparable accuracy.  $x, t \in \{0, 1\}$*

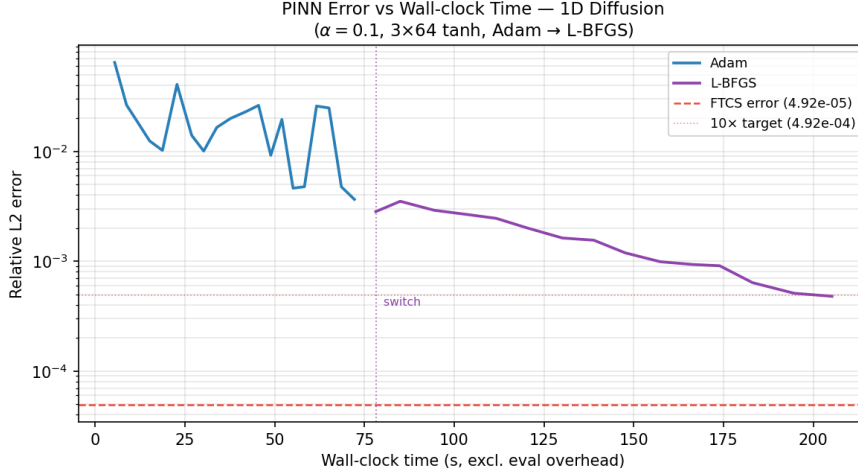


Figure 8: Relative  $L^2$  error against wall-clock time with both the optimiser switching criterion and the error achieved by the FTCS method labelled.

A small PINN  $[3 \times 64]$  with tanh activation requires over  $10,000\times$  the compute time of FTCS to solve the 1D diffusion equation on the available hardware (see Table.4. It should be noted that the difference may not be quite so stark if the PINN were trained on more capable, GPU-accelerated hardware. Nevertheless, numerically solving simple PDEs is much more efficient than the application of PINNs.

### 3.2.2 The Inverse Problem

Clearly, PINNs are a poor choice for solving PDEs such as Eq. (17); however, the ability of a PINN to learn any parameter, not just its own weights and biases, has clear applications. To explore this, a PINN is applied in the inverse problem where the aim is to recover the diffusivity from some noisy data. We implement this by including  $\alpha$  as a parameter in the network:

```
alpha_param = nn.Parameter(torch.tensor(float(ALPHA_INIT)))
...
opt = torch.optim.Adam(
    list(model.parameters()) + [alpha_param], lr=LEARNING_RATE)
```

where ALPHA\_INIT is an initial guess and intermediate code has been omitted for brevity. The diffusivity is recovered numerically by wrapping the FTCS solver in `scipy.optimize.minimize`.

The training data is sampled at 500 points from the FTCS<sup>1</sup> solution and Gaussian noise added to the data in order to mimic ‘real data’. This gave much more promising results for the PINN. With both data losses and the IC/BC/PDE losses to learn from, the PINN recovered  $\alpha$  remarkably well; reducing the runtime gap between itself and the FTCS solver dramatically.

For solving the inverse problem, the PINN is now only 18x slower than the FTCS inverse but perhaps more importantly, it is able to recover  $\alpha$  to much greater accuracy. We see this qualitatively in the neural network’s training history (fig.9) where it converges almost exactly on  $\alpha = 0.5$

<sup>1</sup>The training data could have been sampled from the exact solution, but a precise numerical solution was used to demonstrate the applicability for PDEs without analytical solutions

Table 2: Inverse Problem: PINN vs FTCS –Recovering Diffusion Coefficient,  $\alpha_{\text{init}} = 0.1$   $\alpha_{\text{true}} = 0.5$   $\sigma_{\text{noise}} = 0.2$

| Method          | Architecture       | Final $\alpha$      | Rel. Error | Time to FTCS $\alpha$ (s) | Time to $\alpha \geq 0.495$ (s) |
|-----------------|--------------------|---------------------|------------|---------------------------|---------------------------------|
| PINN (Adam)     | $3 \times 64$ tanh | $0.4870 \pm 0.0032$ | 2.59%      | $27.24 \pm 2.10$          | $45.88 \pm 1.07$                |
| FTCS + L-BFGS-B | scipy scalar opt.  | 0.4633              | 7.33%      | 1.52                      | N/A                             |

*Note: Standard deviations are calculated for PINN values trained over 3 seeds. These were not calculated for FTCS runs since it is deterministic.*

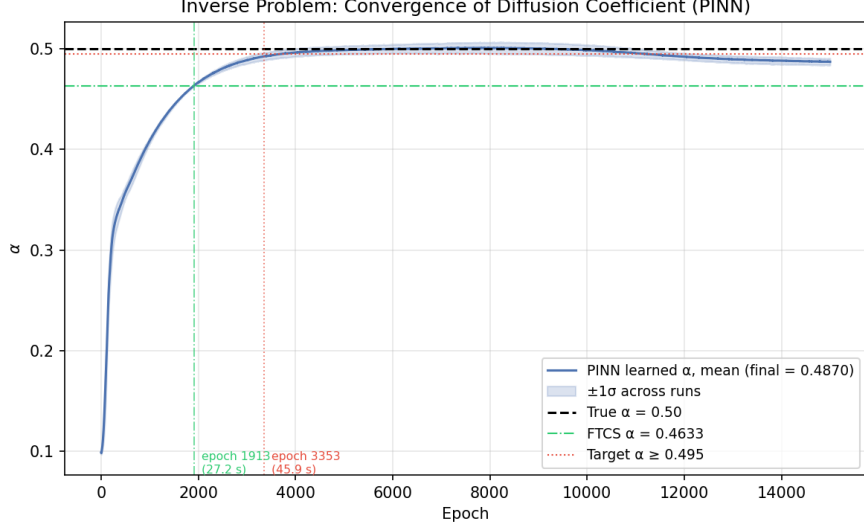


Figure 9: Training history: Convergence of alpha over training epochs (Adam optimised). The training targets  $\alpha = \alpha_{\text{FTCS}}$  and  $\alpha = 0.495$  are included.

### 3.2.3 1D Burgers' equation

We now consider a more complex case, the non-linear 1D Burger's equation. The Burgers' equation finds applications in fluid dynamics, non-linear acoustics and even traffic management.

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2} \quad (21)$$

Where  $\nu$  is the viscosity. Inspired by Raissi et al. [4] we consider the initial condition:

$$u(x, 0) = -\sin(\pi x) \quad (22)$$

From equation.21 we see that the fluid has a velocity proportional to its displacement. Thus, fluid in the region  $x < 0$  has a velocity  $v > 0$  and vice versa, creating a shock at the origin as the fluid collides with itself. Such a shock region may be difficult for a PINN to learn due to spectral bias; we explore this now.

Equation.21 can be solved exactly using the Cole-Hopf transformation [15], [16], however, this is not implemented here due to its unnecessary complexity. We instead utilise another numerical method to solve the Burgers equation, the Method of Lines (MOL), following ref.[17].



We begin by discretising the spatial derivatives in eq.21. The second-order derivative has a similar treatment to that in the diffusion equation:

$$\left(\frac{\partial^2 u}{\partial x^2}\right)_i \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{(\Delta x)^2} \quad (23)$$

However, the first-order 'advective-like' term describes a directional flow of information, so the same central difference is not really appropriate. We instead implement the upwind scheme such that for positive  $u$  (i.e. travelling rightwards), we take the difference between the  $i$  and  $i-1$  points, and vice versa for negative  $u$ . This ensures the derivative approximation is not implementing non-causal information.

$$\left(u \frac{\partial u}{\partial x}\right)_i \approx \begin{cases} u_i \left(\frac{u_i - u_{i-1}}{\Delta x}\right) & \text{if } u_i > 0 \\ u_i \left(\frac{u_{i+1} - u_i}{\Delta x}\right) & \text{if } u_i < 0 \end{cases} \quad (24)$$

The upwind scheme is straightforward to implement and was done so in this project using (see `burgers_1d_mol` in `src/solvers.py`):

```
backward = (u[1:-1] - u[:-2]) / dx
forward = (u[2:] - u[1:-1]) / dx
advection = (np.maximum(u_centre, 0.0) * backward
            + np.minimum(u_centre, 0.0) * forward)
```

Now combining eq.21, eq.23 and eq.24 we have an ODE in time for each interior grid point:

$$\frac{d\mathbf{u}}{dt} = \mathbf{f}(\mathbf{u}, t) \quad (25)$$

which is readily solved using `scipy.integrate.solve_ivp`. Now, let us first look at the solutions produced by MOL and a PINN (at  $\nu = 0.01/\pi$ ). We also plot the absolute difference  $|u_{PINN} - u_{MOL}|$  to quantify PINN performance in different regions.

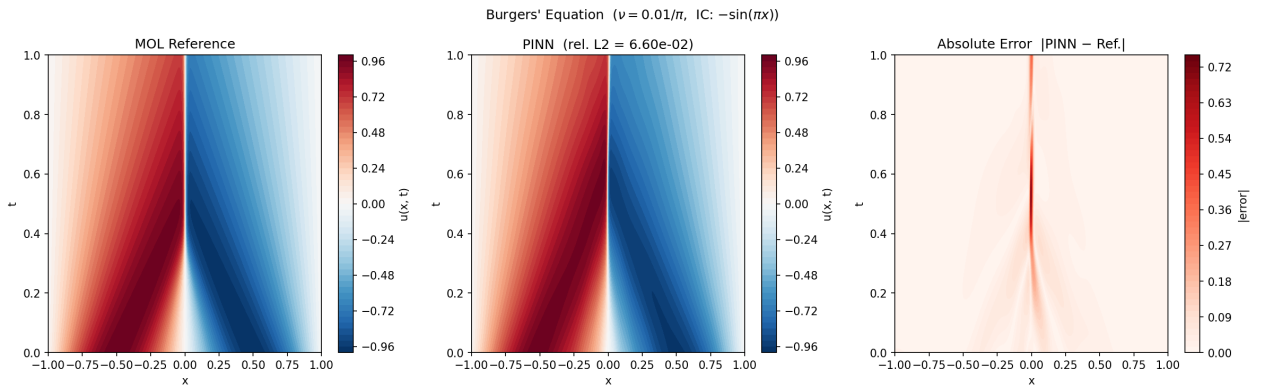


Figure 10: Contour plots of solutions to the 1D burgers equation in the domain  $t \in \{0, 1\}$ ,  $x \in \{-1, 1\}$ . Tanh activated PINN trained across 10,000 + 1000 epochs. The absolute difference of the two solutions is given in the third plot.

Figure.10 shows that the PINN struggles most with the shock formation ( $x = 0$  and  $t \gtrsim 0.4$ ), understood as another manifestation of spectral bias. We now apply the SIREN architecture

(see section 2.4) to investigate whether its high-frequency representation capacity improves resolution of the near-shock region across a viscosity sweep.

The standard initialisation of  $\omega_0 = 30$ , prescribed by Sitzmann et al. [11] for image fitting applications, produced trivial collapse at high viscosity and unstable training across much of the sweep. We attribute this to the mismatch between the image-fitting context for which  $\omega_0 = 30$  was calibrated and the lower-frequency content of Burgers’ solutions; even at low viscosity, the solution contains far fewer spatial modes than an image. Following a parameter search over  $\omega_0 \in \{1, 5, 10, 30\}$ , we adopted  $\omega_0 = 1$ , which produced stable convergence across the majority of the sweep. L2RE and max absolute error (against MOL reference) for a tanh-activated PINN and a SIREN across the viscosity sweep are presented in fig. 11.

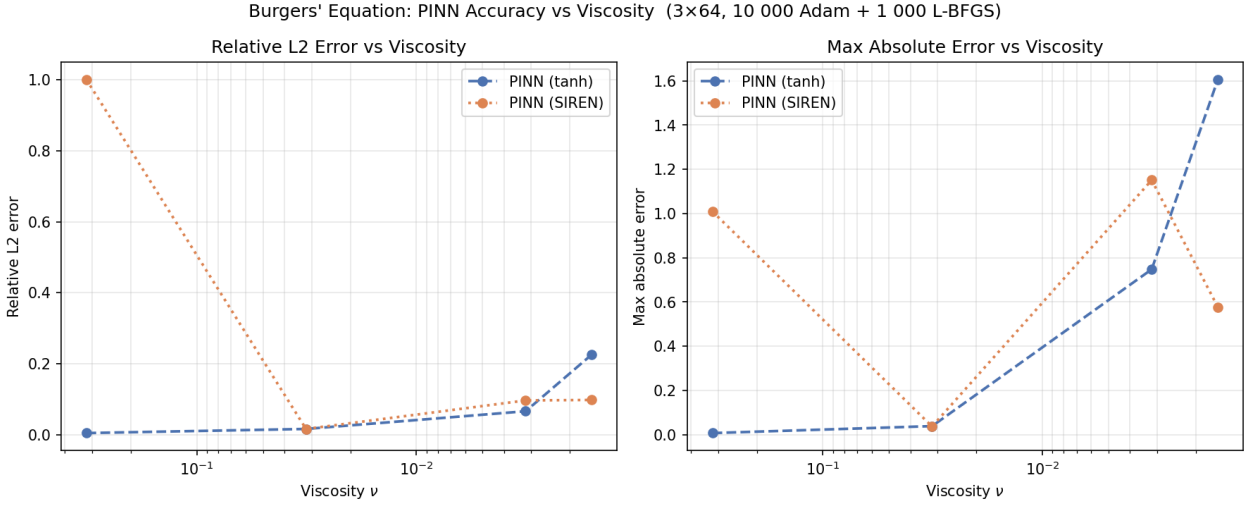


Figure 11: L2RE and Max absolute error against MOL against viscosity,  $\nu \in \frac{1}{\pi}\{1.0, 0.1, 0.01, 0.005\}$ . For the solution grids at each viscosity, see appendix C

While we were limited in the range of viscosity that we could sweep due to instability at either end of the range, we argue that (after the PINN recovers from trivial collapse for large  $\nu$ ) the SIREN produces shallower gradients in both L2RE and absolute error. Indeed, the SIREN performs better at the lowest viscosity (highest spectral content and shock), suggesting that SIRENs are better at comprehending higher-frequency content than traditional PINNs. Clearly, there is more work to be done to demonstrate this definitively; however, compute time and program stability constrain both the testable viscosity domain and the number of viscosity points, not helped by the SIREN being perhaps improperly optimised. Nevertheless, SIRENs do appear to perform better at comprehending sharper gradients.

### 3.2.4 Curriculum Learning

Finally, we attempt curriculum learning. That is, to train a neural network in an easier-to-learn regime, save its trained parameters and then train the PINN in a harder-to-learn regime. The idea is that the neural network gains some ‘understanding’ from the easier regime, so it then has a ‘warm start’ to help it understand a more difficult regime.

We implement this by training a PINN on the 1D Burger’s equation with  $\nu = 0.01/\pi$  for a SIREN and tanh network. The same viscosity sweep is performed as in section. 3.2.3.

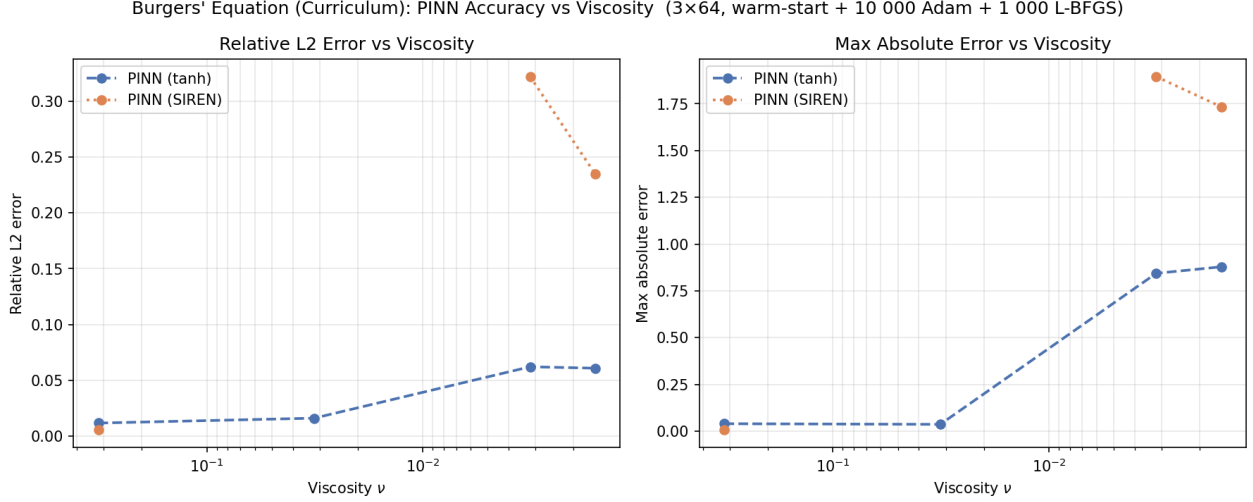


Figure 12: L2RE and Max absolute error against MOL against viscosity,  $\nu \in \frac{1}{\pi}\{1.0, 0.1, 0.01, 0.005\}$  for pre-trained neural networks. For curriculum solutions at each viscosity, see appendix

The results in figure.12 present several interesting points. The SIREN result at  $\nu = 0.01/\pi$  is omitted due to training divergence (NaN). The cause of the divergence is unclear; a possible explanation is related to the PINN already being close to the loss minima, tripping up the Adam optimiser to take large steps in the wrong direction and diverging. It is not clear why the SIREN diverges while the tanh-activated PINN does not.

On the other hand, curriculum learning allowed the SIREN to correctly produce the solution to the low-viscosity case, and to greater accuracy than tanh. At all viscosities, the curriculum-trained tanh-activated PINN produced a lower error (both L2RE and max Absolute error) than it did in the initial, non-curriculum case; decreasing from 0.2 L2RE to 0.05 L2RE at its worse in the low viscosity regime.

In the two lowest viscosity regimes, the curriculum-trained SIREN produced an unusual artefact in which the shock region deviates leftward, inflating the errors for these solutions. As in the original sweep, the grid of solutions and errors is given in Appendix C. The reason for such a deviation is not clear.

## 4 Conclusion

This project has evaluated Physics-Informed Neural Networks across a hierarchy of differential equations, from simple ordinary differential equations to the nonlinear Burgers' equation. The results provide a nuanced assessment of when and how PINNs are useful in computational physics.

For standard forward problems, PINNs are not competitive with traditional numerical methods. The 1D diffusion equation timing comparison showed the PINN approximately four orders of magnitude slower than FTCS at comparable accuracy; a gap that GPU acceleration would narrow but not close. The range of tested problems in this report is small, so we cannot exclude the possibility that PINNs may be advantageous for more complex where traditional methods struggle (e.g. high-dimensional PDEs, irregular geometries).

The picture changes for inverse problems. When recovering an unknown diffusion coefficient from sparse noisy data, the PINN closed most of the run-time gap (only  $18\times$  slower than FTCS wrapped in `scipy.minimise`) and recovered the diffusivity to higher accuracy. The framework’s natural integration of physics constraints with observational data is its genuine strength: no nested optimisation loop is required. This is where PINNs have real application potential in recovering unknown parameters from measurements, such as aerodynamics applications or subsurface geophysics.

Architecture and activation function selection proved consequential. The SHO frequency sweep demonstrated spectral bias — the tendency of networks to learn low-frequency components first, with high frequencies either learned poorly or causing trivial collapse to  $\hat{u} = 0$ . The Burgers’ equation viscosity sweep extended this to the nonlinear regime, revealing a viscosity-dependent crossover: tanh activations perform best for smooth high-viscosity solutions, while SIREN achieves better resolution of the near-shock region at low viscosity. This is consistent with the spectral content of the solution in each regime. Further optimisation of the SIREN is possible; we merely ‘scratched the surface’.

Curriculum learning was demonstrated to be significantly beneficial to PINN solution accuracy, particularly for the tanh-activated PINN. One would expect to conclude that a similar benefit can be had for the SIREN implementation, and indeed this was observed at low viscosity, but this report does not present an explanation for the observed shock deviation at low viscosity for curriculum-trained SIRENs.

Several limitations and avenues for future work emerge naturally. The Burgers’ viscosity sweep was constrained by training instability at both ends of the range; a more comprehensive study would benefit from adaptive collocation point sampling near the shock. The SIREN architecture was found to underperform in some regimes, possibly due to suboptimal hyperparameter choices not fully explored in this project.

The broader takeaway is that PINNs are not a replacement for traditional numerical methods, but a complement to them. They occupy a specific niche, inverse problems, data assimilation, and scenarios with incomplete or unconventional problem specifications, where the flexibility of the framework outweighs its computational cost. In their domain of strength, PINNs demonstrate impressive potential. Outside that domain, the established methods remain superior.

## A 1D Diffusion timing methodology

Table 3: 1D Diffusion Equation: FTCS vs PINN Timing Comparison ( $\alpha = 0.1$ ,  $x \in [0, 1]$ ,  $t \in [0, 1]$ , mode  $n = 1$ )

| Method      | Architecture                  | Rel. L2 Error         | Wall-clock Time (s) |
|-------------|-------------------------------|-----------------------|---------------------|
| FTCS        | $\Delta x = 0.01$ , $r = 0.4$ | $4.92 \times 10^{-5}$ | 0.0286              |
| PINN (Adam) | $3 \times 64$ tanh            | $3.08 \times 10^{-3}$ | 735.2220            |

*PINN trained for 30,000 epochs (time excludes evaluation overhead). PINN is  $25,715.6\times$  slower than FTCS at equivalent accuracy. Note: PINN did not reach FTCS accuracy within the epoch cap.*

Table 4: Computational time comparison for solving the 1D diffusion equation with both the FTCS method and a PINN with the Adam–L-BFGS switching condition and  $10\times$  FTCS error condition.

| Method | Architecture               | Rel. L2 Error         | Wall-clock Time (s) |
|--------|----------------------------|-----------------------|---------------------|
| FTCS   | $\Delta x = 0.01, r = 0.4$ | $4.92 \times 10^{-5}$ | 0.0161              |
| PINN   | $3 \times 64 \tanh$        | $4.79 \times 10^{-4}$ | 205.2214            |

*Note: PINN training utilized 4,000 Adam + 750 L-BFGS steps (time excludes evaluation overhead). PINN is  $12,782\times$  slower than FTCS at comparable accuracy.  $x, t \in \{0, 1\}$*

The initial approach did not produce rigorous data since the runtime could vary significantly if the Adam training epochs had a ‘fortunate’ error dip below the criterion. It also was not representative of the error achievable by the neural network. Indeed, the second method could be refined further and the location of the switch from Adam to L-BFGS is clearly a variable that could be optimised in and of itself. This was not necessary to convey the disparity between the computational runtime of numerical solvers and physics-informed neural networks.

In all measurements of PINN timing, the time increase due to evaluation every 200 or 50 epochs is subtracted from the final measured time. That is, all times for this experiment were reported excluding the evaluation overhead.

## B Derivation of the Newton update[2]

The Newton update used as the basis for quasi-Newton methods such as L-BFGS arises from a second-order Taylor expansion of the loss about the current parameters  $\theta_0$ :

$$\mathcal{L}(\theta) \approx \mathcal{L}(\theta_0) + \nabla \mathcal{L}^\top (\theta - \theta_0) + \frac{1}{2}(\theta - \theta_0)^\top H (\theta - \theta_0) \quad (26)$$

where  $H$  is the Hessian matrix and  $^\top$  denotes the transpose. Setting the derivative with respect to  $\theta$  to zero locates the minimum of this local quadratic approximation:

$$\frac{\partial \mathcal{L}}{\partial \theta} = 0 + \nabla \mathcal{L} + H(\theta - \theta_0) = \nabla \mathcal{L} + H(\theta - \theta_0) = 0 \quad (27)$$

where we have used the identity  $\frac{\partial}{\partial \mathbf{x}} (\mathbf{x}^\top A \mathbf{x}) = (A + A^\top) \mathbf{x}$  and the symmetry of the Hessian to take the derivative. We thus have:

$$\nabla \mathcal{L} + H(\theta - \theta_0) = 0 \implies \theta = \theta_0 - H^{-1} \nabla \mathcal{L} \quad (28)$$

The factor  $H^{-1}$  rescales the gradient by the local curvature: steep directions receive small steps and shallow directions receive large steps.

# C 1D Burger's Equation Viscosity sweep, solution and error grid

Burgers' Equation: Space-Time Solutions and Errors Across Viscosities

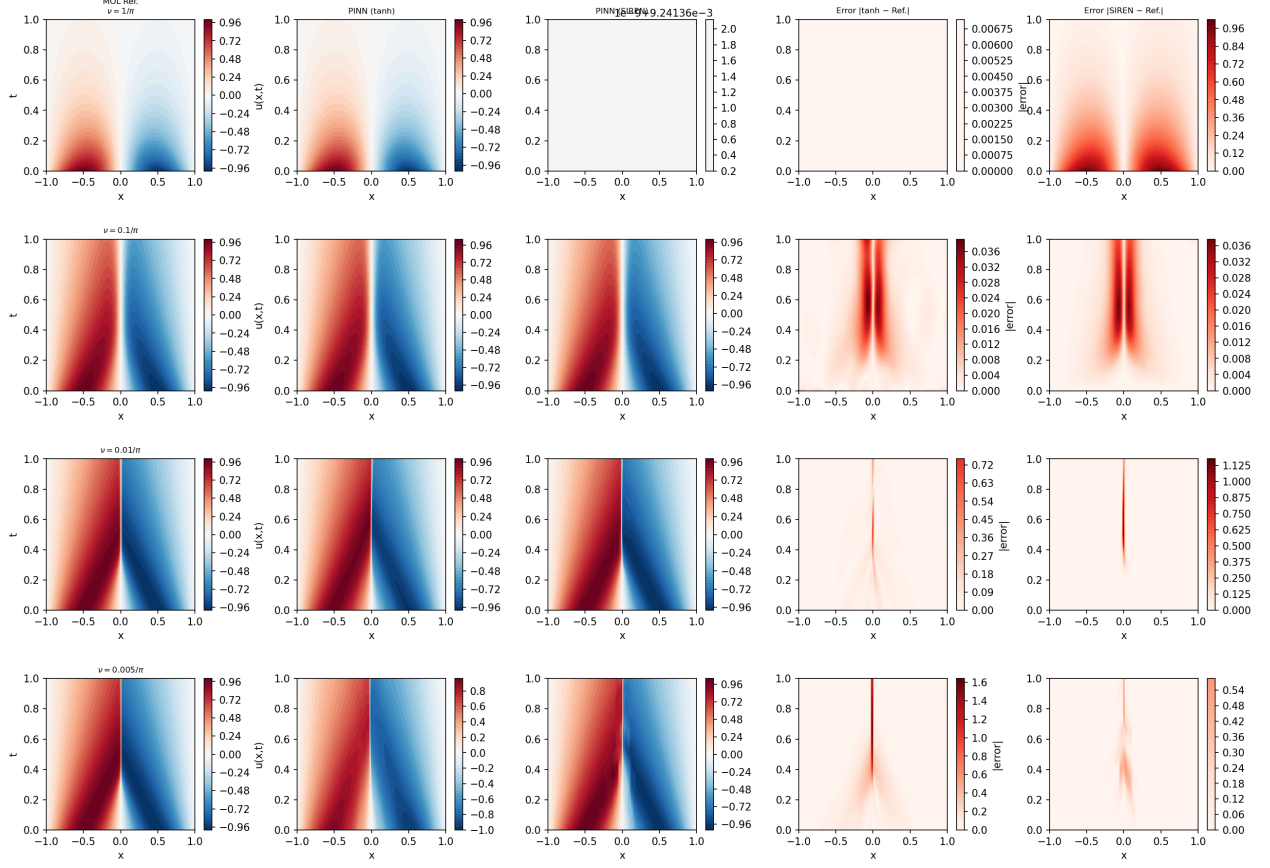


Figure 13: Solution and error grid for viscosity sweep solutions, tanh and SIREN networks

## D 1D Burger's Equation curriculum viscosity sweep, solution and error grid

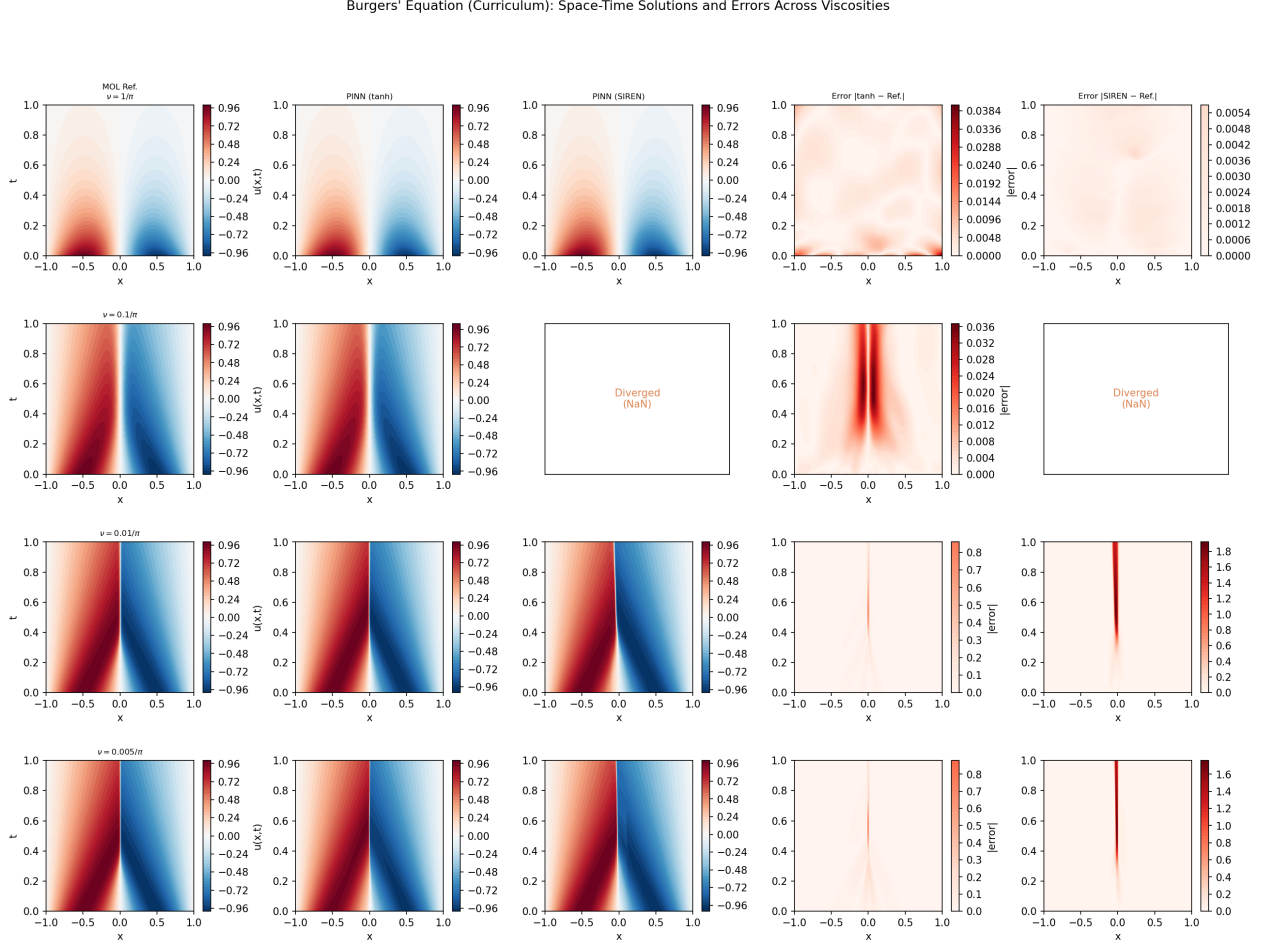


Figure 14: Solution and error grid for viscosity sweep solutions for curriculum taught tanh and SIREN networks

## References

- [1] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006, pp. 225–284.
- [2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, Massachusetts: The MIT Press, 2016. [Online]. Available: <https://www.deeplearningbook.org/>.
- [3] S. Lee, *Sangheon lee*, Aug. 2024. [Online]. Available: <https://lazyjobseeker.github.io/en/posts/physics-informed-neural-network-tutorials/>.
- [4] M. Raissi, P. Perdikaris, and G. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019, ISSN: 0021-9991. DOI: <https://doi.org/10.1016/j.jcp.2018.10.045>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0021999118307125>.
- [5] PyTorch, *Pytorch documentation — pytorch 2.7 documentation*, 2024. [Online]. Available: <https://docs.pytorch.org/docs/stable/index.html>.
- [6] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization*, 2017. arXiv: [1412.6980](https://arxiv.org/abs/1412.6980). [Online]. Available: <http://arxiv.org/abs/1412.6980>.



- [7] D. C. Liu and J. Nocedal, “On the limited memory BFGS method for large scale optimization,” *Mathematical Programming*, vol. 45, no. 1–3, 1989. DOI: [10.1007/bf01589116](https://doi.org/10.1007/bf01589116).
- [8] J. Nocedal and S. J. Wright, *Numerical Optimization* (Springer Series in Operations Research and Financial Engineering), 2nd ed. New York: Springer, 2006, p. 183.
- [9] P. Rathore, W. Lok, J. Yang, Z. Wang, and M. Udell, “Challenges in training PINNs: A loss landscape perspective,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024. arXiv: [2402.01868](https://arxiv.org/abs/2402.01868).
- [10] N. H. Dashtbayaz, G. Farhani, B. Wang, and C. X. Ling, *Physics-Informed Neural Networks: Minimizing residual loss with wide networks and effective activations*, 2024. arXiv: [2405.01680](https://arxiv.org/abs/2405.01680) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2405.01680>.
- [11] V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell, and G. Wetzstein, *Implicit neural representations with periodic activation functions*, 2020. arXiv: [2006.09661](https://arxiv.org/abs/2006.09661) [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2006.09661>.
- [12] V. Sitzmann, *Siren: Implicit neural representations with periodic activation functions, github repository*, 2025. [Online]. Available: [https://github.com/vsitzmann/siren/blob/master/explore\\_siren.ipynb](https://github.com/vsitzmann/siren/blob/master/explore_siren.ipynb).
- [13] S. Khodakarami, V. Oommen, N. A. Daryakenari, M. Beekenkamp, and G. E. Karniadakis, *Spectral bias in physics-informed and operator learning: Analysis and mitigation guidelines*, 2026. arXiv: [2602.19265](https://arxiv.org/abs/2602.19265).
- [14] D. L. Whalley, *Computational physics — forward time central space*, Accessed 24 Apr. 2026, 2026. [Online]. Available: [https://nu-cem.github.io/Computational\\_Physics/notebooks/FTCS\\_method.html](https://nu-cem.github.io/Computational_Physics/notebooks/FTCS_method.html).
- [15] J. D. Cole, “On a quasi-linear parabolic equation occurring in aerodynamics,” *Quarterly of Applied Mathematics*, vol. 9, no. 3, Oct. 1951. DOI: <https://doi.org/10.1090/qam/42889>.
- [16] E. Hopf, “The partial differential equation  $u_t + u u_x = x x$ ,” *Communications on Pure and Applied Mathematics*, vol. 3, no. 3, pp. 201–230, Sep. 1950. DOI: <https://doi.org/10.1002/cpa.3160030302>.
- [17] J. Biazar, Z. Ayati, and S. Shahbazi, “Solution of the burgers equation by the method of lines,” *American Journal of Numerical Analysis*, vol. 2, pp. 1–3, Jan. 2014.